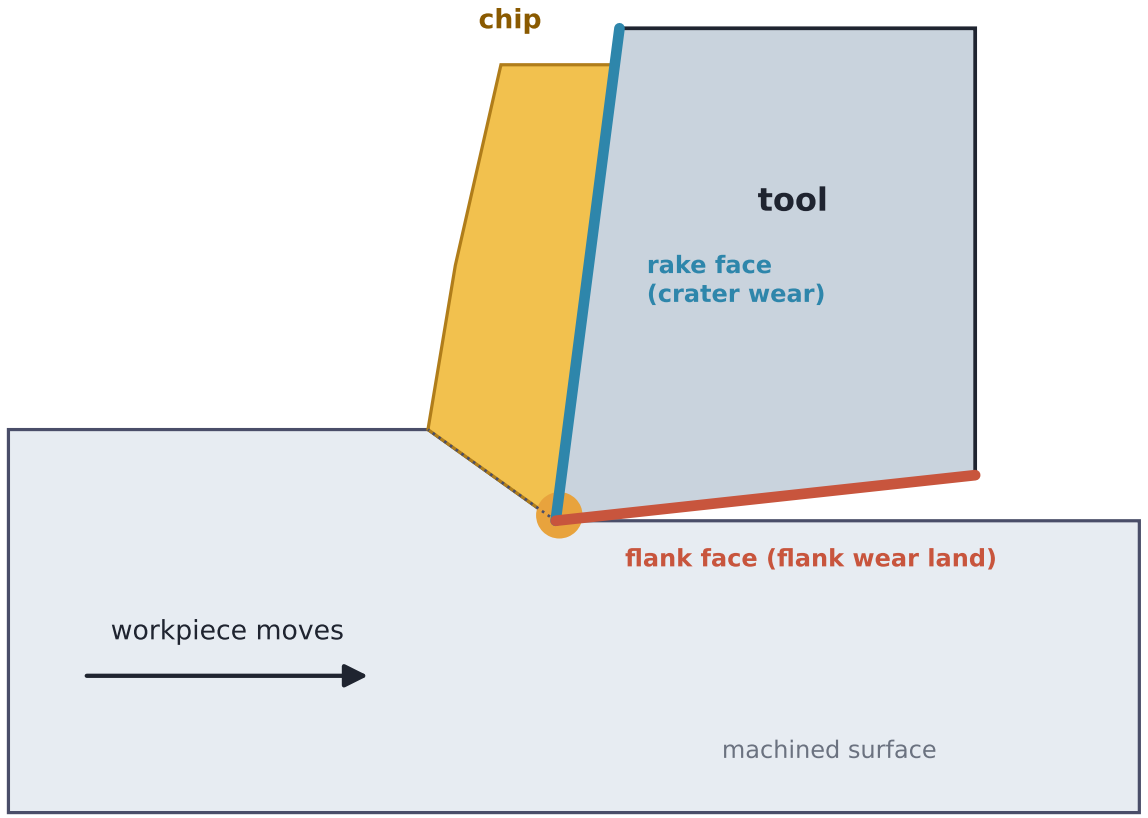
THE PROBLEM

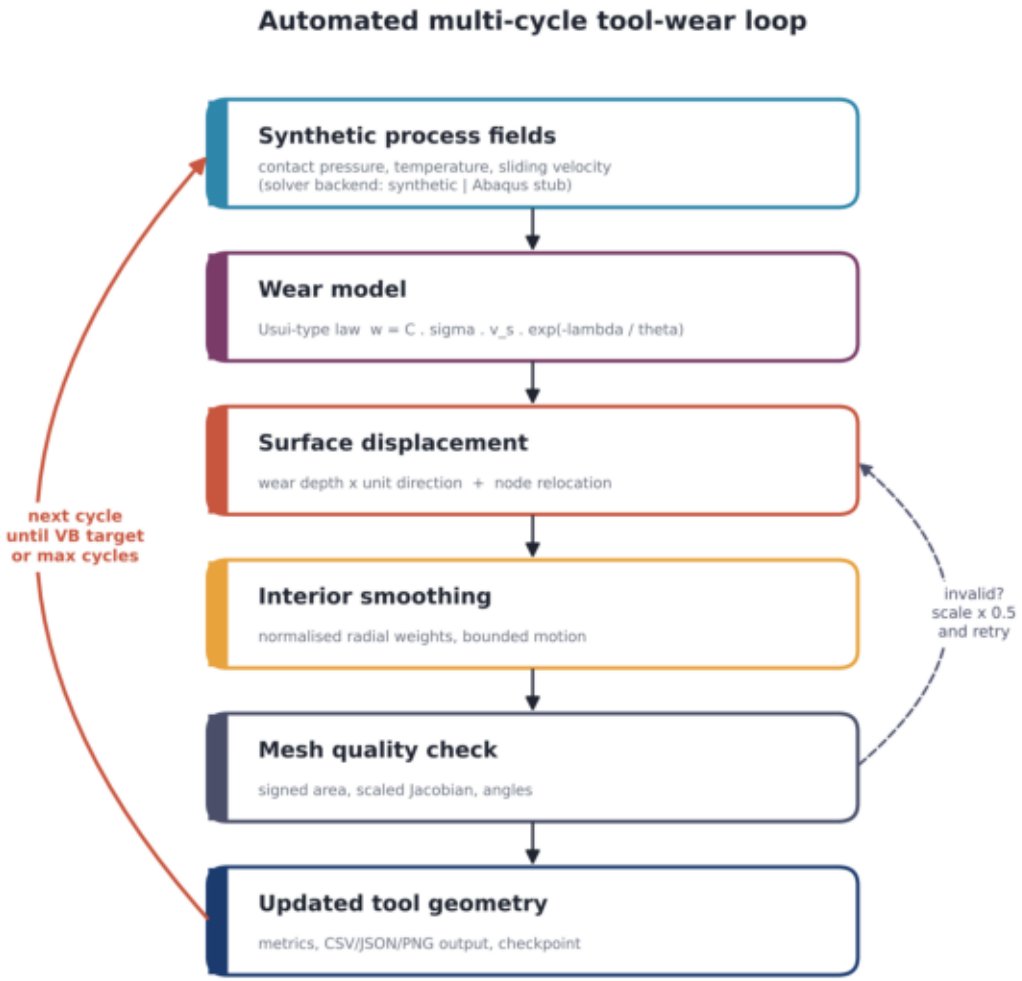
Automating incremental tool-wear prediction

- Tool wear changes the cutting-edge geometry - and with it forces, temperatures and part quality.
- Wear experiments are slow and expensive; FEM can predict wear, but only if the geometry is updated cycle by cycle.
- Doing that by hand (simulate, extract fields, move nodes, fix the mesh, re-run) does not scale.
- This project automates the full loop in Python - solver-free, so anyone can run and inspect it.



WORKFLOW

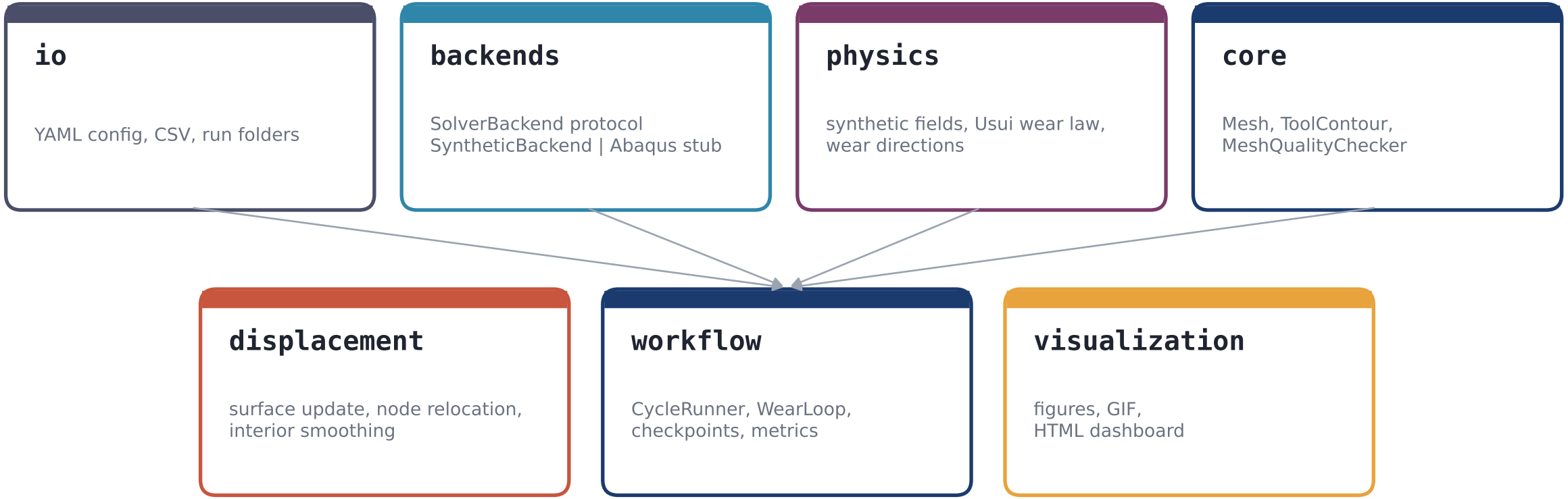
One automated loop, repeated until a wear criterion



- Process fields on the tool surface come from a pluggable backend (synthetic by default).
- A literature wear law turns local pressure, temperature and sliding speed into a wear rate.
- Surface nodes recede; interior nodes follow; the mesh is checked and the step halved if needed.
- Each cycle writes CSV/JSON/PNG and a checkpoint - interrupted runs resume bit-for-bit.

DESIGN

Software architecture: physics, geometry and solver kept apart



src/tool_wear_demo/ - standard Python (NumPy, Matplotlib, PyYAML); no commercial solver needed

74 unit + integration tests, GitHub Actions on Python 3.11 / 3.12, deterministic seeded runs, CLI + examples

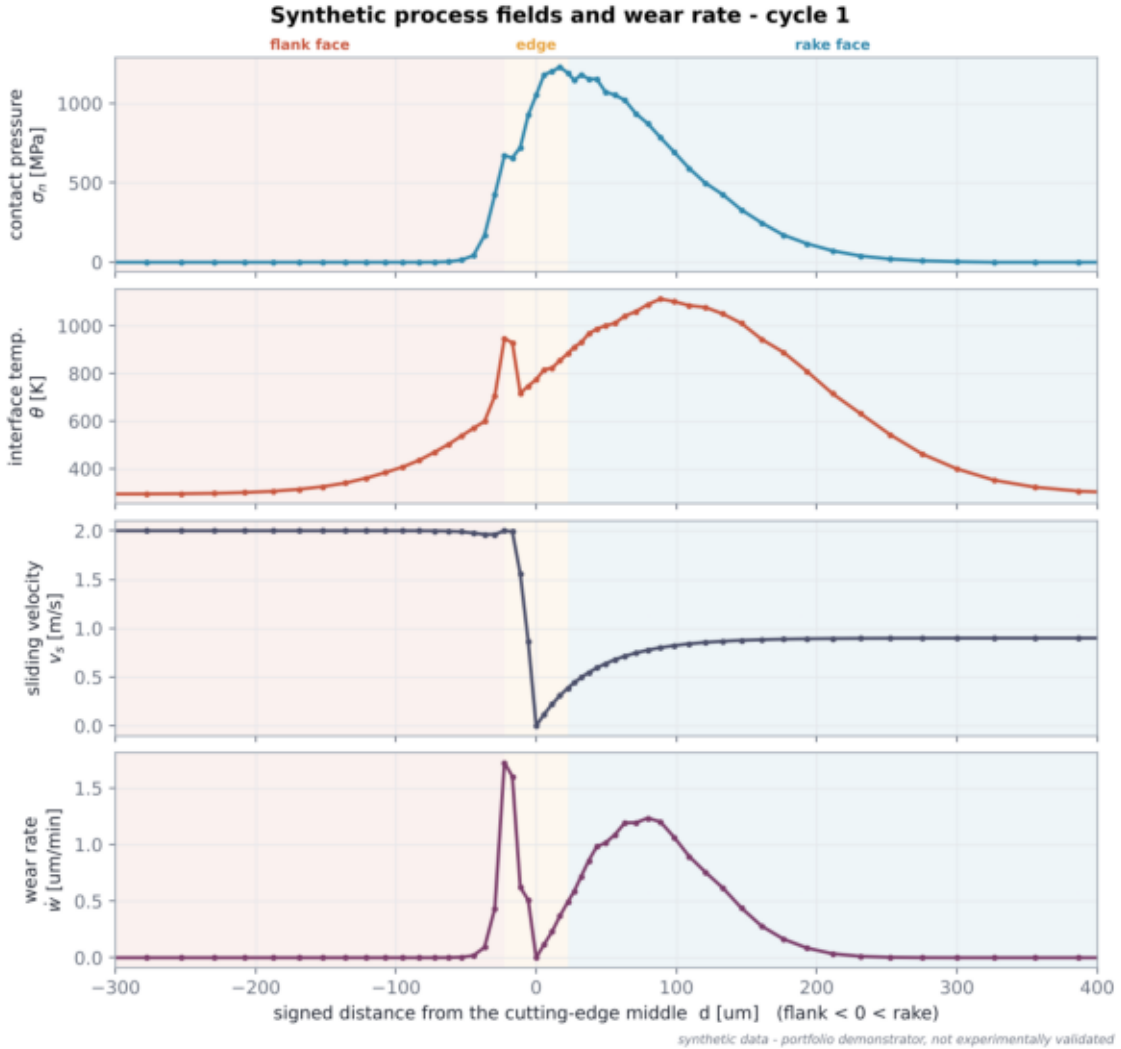
PHYSICS

Wear law and synthetic process fields

$$\dot{w} = C\,\sigma_n\,v_s\,\exp\left(-\frac{\lambda}{\theta}\right)$$

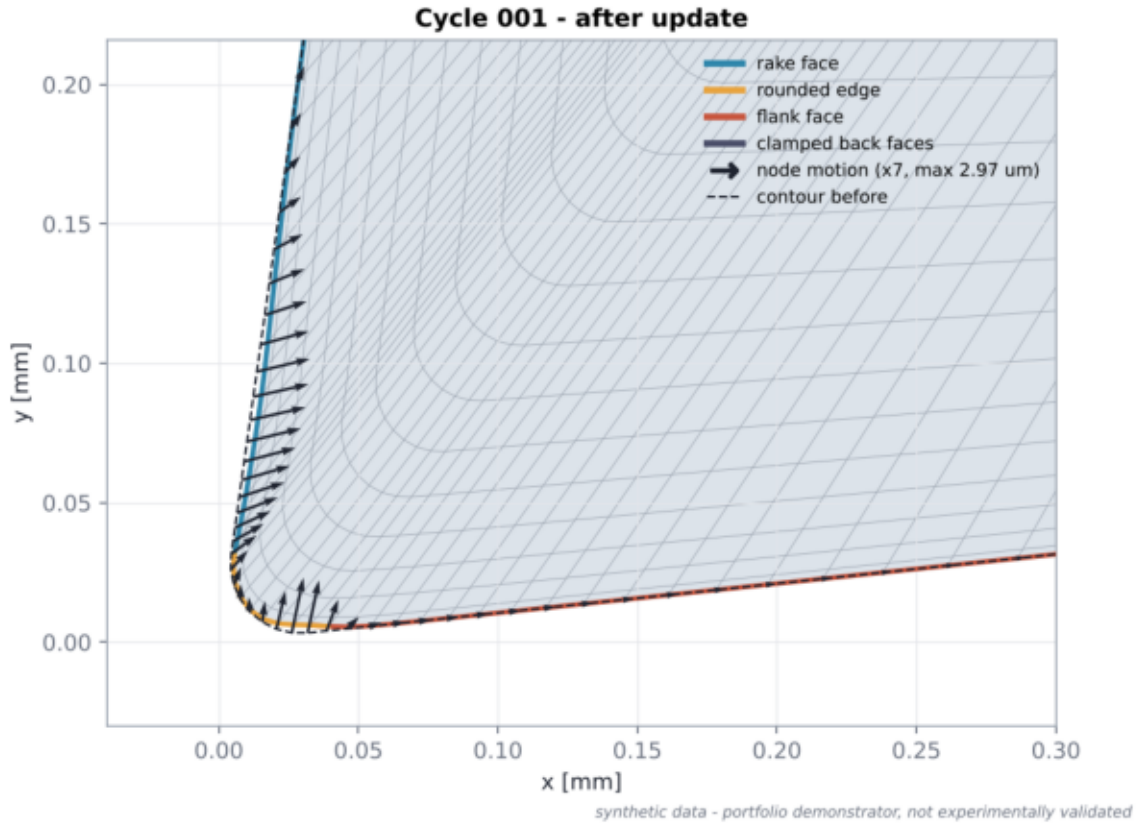
Usui-type wear law as used for FEM wear estimation
(Xie et al., Wear 258, 2005)

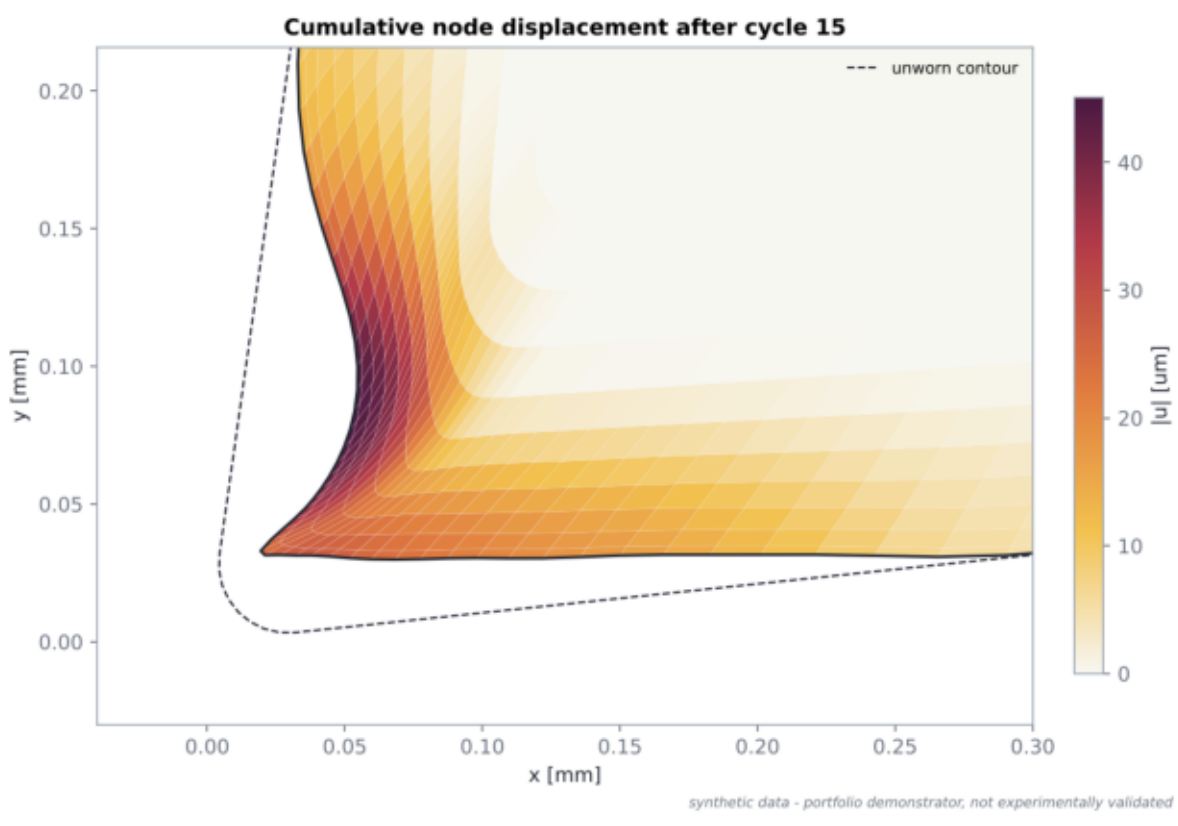
- Pressure peaks at the edge, temperature behind it on the rake face (crater zone).
- Sliding speed: zero at the stagnation point, chip speed on the rake, cutting speed on the flank.
- Flank contact is geometric: nodes within an elastic-recovery gap of the lowest point rub - so the wear land grows with wear.
- Demo constants are illustrative; the time step is auto-scaled (numerical control, not calibration).



NUMERICS

Geometry update that keeps the mesh healthy



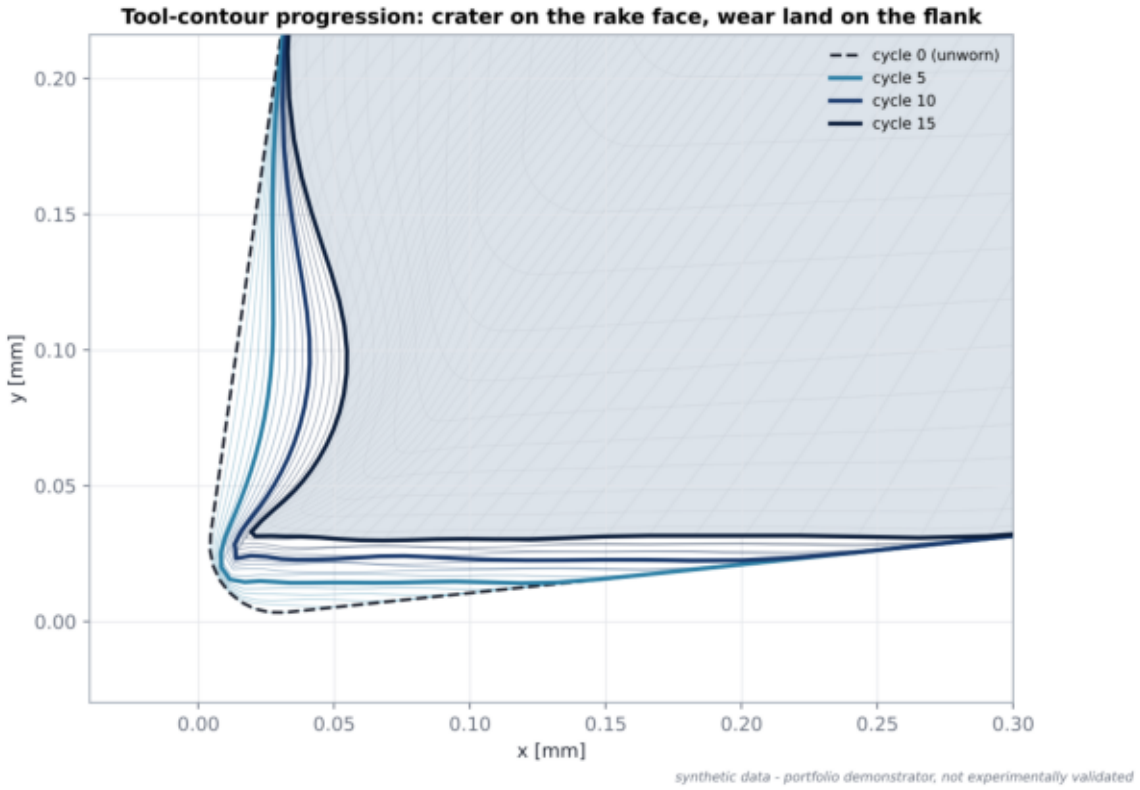


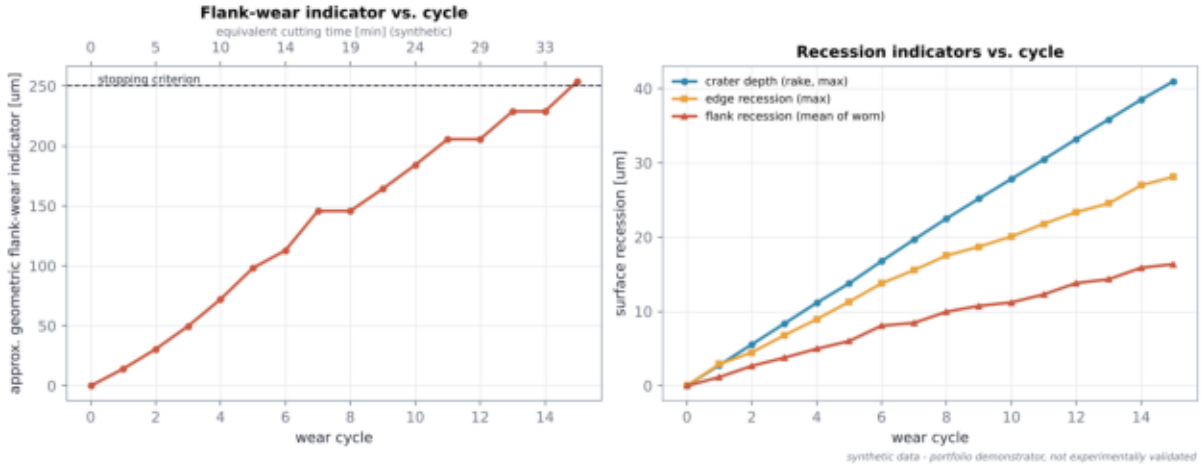
Wear depth x direction (flank: normal to cutting velocity; rake: surface normal) -> nodes slide back to their original spacing along the worn profile

-> interior follows via normalised, tapered inverse-distance weights (|u_interior| <= max |u_surface|) -> scaled-Jacobian check, step x 0.5 on failure

RESULTS

Multi-cycle result: crater and flank wear land emerge





15 cycles | 36 min equivalent cutting time (synthetic)

flank-wear indicator 254 um | crater depth 41 um

stopped by the flank-wear criterion; no inverted elements (min scaled Jacobian 0.19)

TAKEAWAYS

Engineering value and skills demonstrated

Python & automation

- modular package, CLI, config-driven runs
- checkpoint / resume, per-cycle outputs

FEM workflow thinking

- solver -> fields -> wear -> geometry -> solver
- backend interface ready for a solver adapter

Manufacturing physics

- orthogonal cutting, Usui wear law
- crater vs. flank wear, VB-type indicator

Numerical geometry

- contour normals, node relocation
- bounded smoothing, Jacobian-based quality

Reproducibility & testing

- seeded synthetic data, 74 tests
- GitHub Actions CI on 3.11 / 3.12

Communication

- figures, GIF, HTML dashboard
- methodology, validation and limits documented

Independently developed demonstrator inspired by published literature. Not an official RWTH Aachen / MTI repository; contains no confidential data or code.